HW9 for CS 6210

You may (and probably should) talk about problems with the each other, with the TA, and with me, providing attribution for any good ideas you might get. Your final write-up should be your own.

```
using LinearAlgebra
```

```
using Plots
```

```
using FFTW
```

```
using LinearMaps
```

```
using IterativeSolvers
```

1. Grandly Galerkin

Recall the problem introduced in HW 8:

$$TU + UT + h^2 W \odot U + h^2 F = 0$$

with right hand side $f_{ij} = x_i^2 + x_j^2$ and potential term $w_{ij} = x_i(1-x_i)x_j(1-x_j)$. Now we can write $W = dd^T$ where $d_i = x_i(1-x_i)$, and letting $D = \mathbf{diag}(d)$, we have

$$R(U) = TU + UT + h^2 DUD + h^2 F = 0.$$

Now consider an approximate solution $\hat{U} = QXQ^T$ where $Q \in \mathbb{R}^{n \times k}$ has orthonormal columns, using the approximation ansatz $Q^T R(\hat{U}) Q = 0$. We note that this is equivalent to Galerkin approximation with the basis $Q \otimes Q$ on the "vec"ed system

$$(T \otimes I + I \otimes T + h^2 D \otimes D)\,\mathbf{vec}(U) = -h^2\,\mathbf{vec}(F)$$

You should be able to write a very similar system for the reduced variable X (do so!).

Write a function

```
solve_schrod_galerkin(Q)
```

to solve this system for a given Q. You may assume k small enough so that direct factorization of the k^2-by-k^2 matrix is not too expensive. For $n = 100$, what is the relative residual $\|R(\hat{U})\|/\|h^2 F\|$ if we choose Q to be an orthonormal basis for the polynomials of degree at most 6 sampled on the n

internal mesh points x_i? What is the relative error (using the iteration from HW 8 to compute a reference solution)?

Hint: It is fine to use Kronecker products to form the small problem, but if it costs you more than $O(n^2k)$ to set things up, you are on the wrong track. Note that if `X` is a $k \times k$ matrix in Julia, then `x = X[:]` is its "vec", and the expression `X[:] = x` can be used to "undo" a vec. You may find it useful to look at the following code for computing the residual.

`schrod_resid (generic function with 1 method)`

```
function schrod_resid(U)
    n, n = size(U)
    h = 1/(n+1)
    T = SymTridiagonal(2*ones(n), -ones(n-1))
    xx = [i/(n+1) for i=1:n]
    F = [x^2 + y^2 for x in xx, y in xx]
    d = xx .* (1.0 .- xx)
    R = T*U + U*T + h^2*(d.*U.*d') + h^2*F
    R, norm(R)/norm(h^2*F)
end
```

You may also want to use the following helper routines from the previous homework for computing your reference solution.

`dst (generic function with 2 methods)`

```
begin
    T1D(n) = SymTridiagonal(2*ones(n), -ones(n-1))
    λ1D(n) = [2*(1-cos(k*π/(n+1))) for k = 1:n]
    dst(v :: Vector{Float64}) = FFTW.r2r(v, FFTW.RODFT00)/sqrt((length(v)+1)*2)
    dst(A :: Matrix{Float64}) = FFTW.r2r(A, FFTW.RODFT00)/sqrt(prod(size(A).+1))/2
end
```

`solve_schrod_galerkin (generic function with 1 method)`

```
function solve_schrod_galerkin(Q)
    n, k = size(Q)
    h = 1 / (n + 1)
    T = SymTridiagonal(2 * ones(n), -ones(n - 1))
    xx = (1:n) / (n + 1)
    d = xx .* (1 .- xx)
    Dmat = Diagonal(d)
    F = [x^2 + y^2 for x in xx, y in xx]

    hatT = Q' * (T * Q)
    hatD = Q' * (Dmat * Q)
    hatF = Q' * (F * Q)

    Ik = Diagonal(ones(k))
    Asmall = kron(hatT, Ik) + kron(Ik, hatT) + h^2 * kron(hatD, hatD)
    bsmall = -h^2 * vec(hatF)

    x = Asmall \ bsmall
    X = reshape(x, k, k)

    X
end
```

The original equation is: $R(U) = TU + UT + h^2 DUD + h^2 F = 0$, where T is the symmetric tridiagonal matrix with 2 on the diagonal and -1 on the off-diagonals, $D = \mathbf{diag}(d)$ with $d_i = x_i(1 - x_i)$, $x_i = i/(n+1)$, $F_{ij} = x_i^2 + x_j^2$, and $h = 1/(n+1)$. We approximate $U \approx \hat{U} = QXQ^T$, where $Q \in \mathbb{R}^{n \times k}$ has orthonormal columns ($Q^T Q = I_k$). The Galerkin condition is $Q^T R(\hat{U}) Q = 0$. Substitute $\hat{U}$:
$R(\hat{U}) = T(QXQ^T) + (QXQ^T)T + h^2 D(QXQ^T)D + h^2 F$. Multiply by Q^T from the left and Q from the right:
$Q^T R(\hat{U}) Q = Q^T T(QXQ^T)Q + Q^T(QXQ^T T)Q + h^2 Q^T D(QXQ^T)DQ + h^2 Q^T FQ$.
Simplify each term: $Q^T T(QXQ^T)Q = (Q^T TQ)X(Q^T Q) = (Q^T TQ)X$,
$Q^T(QXQ^T T)Q = (Q^T Q)X(Q^T TQ) = X(Q^T TQ)$,
$Q^T D(QXQ^T)DQ = (Q^T DQ)X(Q^T DQ)$, $h^2 Q^T FQ$ remains as is.

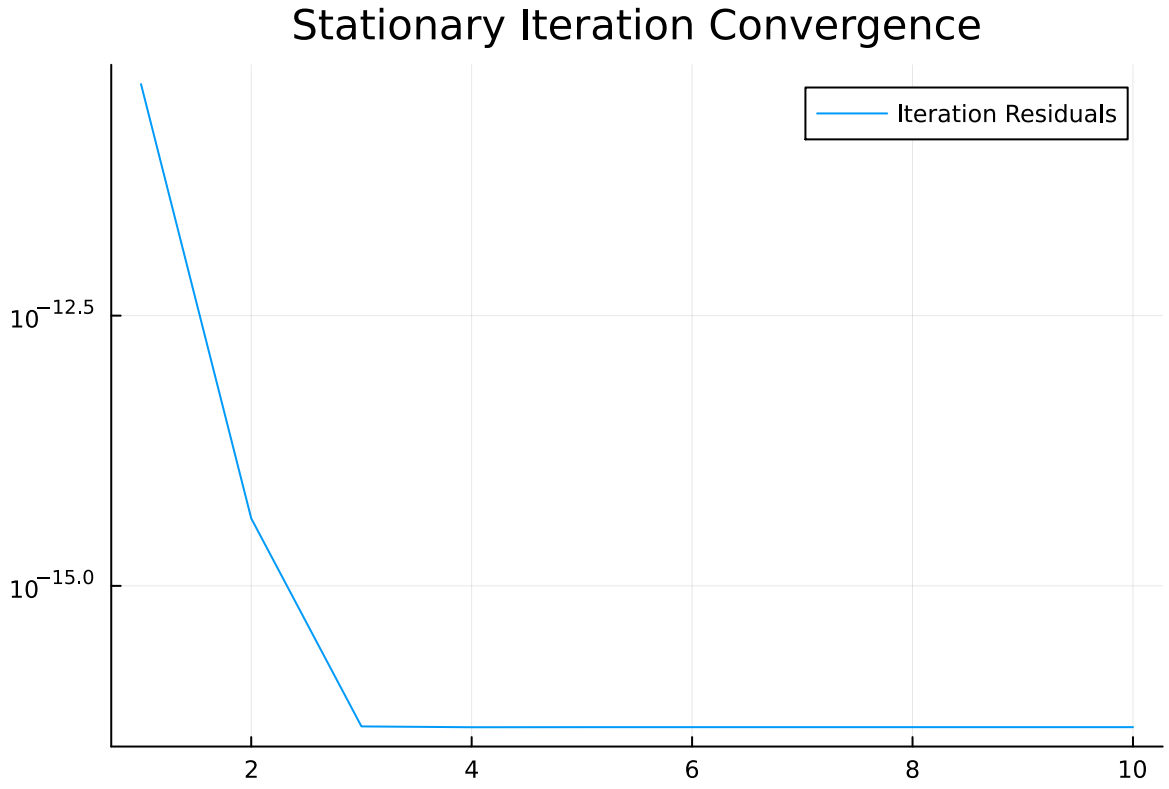
Define $\hat{T} = Q^T TQ$, $\hat{D} = Q^T DQ$, $\hat{F} = Q^T FQ$.

The reduced system is $\hat{T}X + X\hat{T} + h^2 \hat{D}X\hat{D} + h^2 \hat{F} = 0$ Vectorizing (vec operation), this is equivalent to $(\hat{T} \otimes I_k + I_k \otimes \hat{T} + h^2 \hat{D} \otimes \hat{D})\,\mathbf{vec}(X) = -h^2\,\mathbf{vec}(\hat{F})$ where I_k is the $k \times k$ identity. Since k is small, we can form and factor the $k^2 \times k^2$ matrix directly. The setup cost is $O(n^2 k)$: computing $\hat{F} = Q^T(FQ)$ requires $O(n^2 k)$ for FQ (since F is dense $n \times n$) plus $O(nk^2)$ for $Q^T(FQ)$. Other terms ($\hat{T}$, $\hat{D}$) are $O(nk^2)$.

(See below code) For n=100 with an orthonormal basis Q for polynomials of degree at most 6 sampled on the internal mesh points x_i, the relative residual $\|R(\hat{U})\|/\|h^2 F\|$ is 0.06579375259754208. using stationary iteration from HW8 as reference: the final iteration residual is $4.931796969590891e-17$ after 3 steps. The relative error is 0.0008323203896662975.

```
schrodinger_iter (generic function with 1 method)
```

```
function schrodinger_iter(W, F, U, nsteps)
    n = size(F, 1)
    h = 1.0 / (n + 1)
    T = T1D(n)
    lambda = λ1D(n)
    resids = Float64[]
    for step = 1:nsteps
        G = F + W .* U
        tilde_G = dst(G)
        tilde_U = similar(tilde_G)
        for j = 1:n, k = 1:n
            tilde_U[j, k] = -h^2 * tilde_G[j, k] / (lambda[j] + lambda[k])
        end
        U = dst(tilde_U)
        R = T * U + U * T + h^2 * (W .* U + F)
        push!(resids, norm(R) / norm(F))
    end
    U, resids
end
```




```
begin
    # test for n=100, polynomial basis degree <=6 (k=7: degrees 0-6)
    n = 100
    grid_sample(n, f :: Function) = [f(i/(n+1), j/(n+1)) for i=1:n, j=1:n]
    w(x,y) = x*y*(1-x)*(1-y)
    f(x,y) = x^2 + y^2
    W_test = grid_sample(n, w)
    F_test = grid_sample(n, f)

    xx = (1:n) / (n + 1)
    # Vandermonde for degrees 0 to 6
    V = hcat([xx .^ (j-1) for j = 1:7]...)
    Fact = qr(V)
    Q = Matrix(Fact.Q)  # Orthonormal basis

    # Compute Galerkin approximation
    X = solve_schrod_galerkin(Q)
    U_test = Q * X * Q'

    # Relative residual
    _, rel_resid = schrod_resid(U_test)
    println("Relative residual: ", rel_resid)

    # Reference solution via iteration
    U_ref, resids_iter = schrodinger_iter(W_test, F_test, U_test, 10)
    println("Final iteration residual: ", resids_iter[end])

    # Relative error
    rel_err = norm(U_test - U_ref) / norm(U_ref)
    println("Relative error: ", rel_err)

    # iteration residuals for convergence check
    plot(resids_iter, yscale=:log10, label="Iteration Residuals", title="Stationary
    Iteration Convergence")
end
```

```
Relative residual: 0.06579375259754208
Final iteration residual: 4.931796969590891e-17
Relative error: 0.0008323203896662975
```

2. Minor modification

Now consider solving the linear system

$$TU + UT + h^2 W \odot U = -h^2 F$$

where W is sparse. To do this, we will use the CG solver from `IterativeSolvers.jl`. The iteration uses `LinearMaps.jl` to define a linear map object from a function that implements the action of the map on the right hand side when applied to a vector (which we reshape between being a matrix and being a flattened vector as needed for the software task).

```
schrod2db (generic function with 1 method)
```

```
function schrod2db(W)
    n, n = size(W)
    h = 1/(n+1)
    T = SymTridiagonal(2*ones(n), -ones(n-1))
    LinearMap(
        function(U)
            U = reshape(U, n, n)
            Y = T*U + U*T + h^2*(W .* U)
            Y[:]
        end, n*n, issymmetric=true, isposdef=true)
end
```

Throughout this exercise, we will use a demo problem on a 100-by-100 mesh.

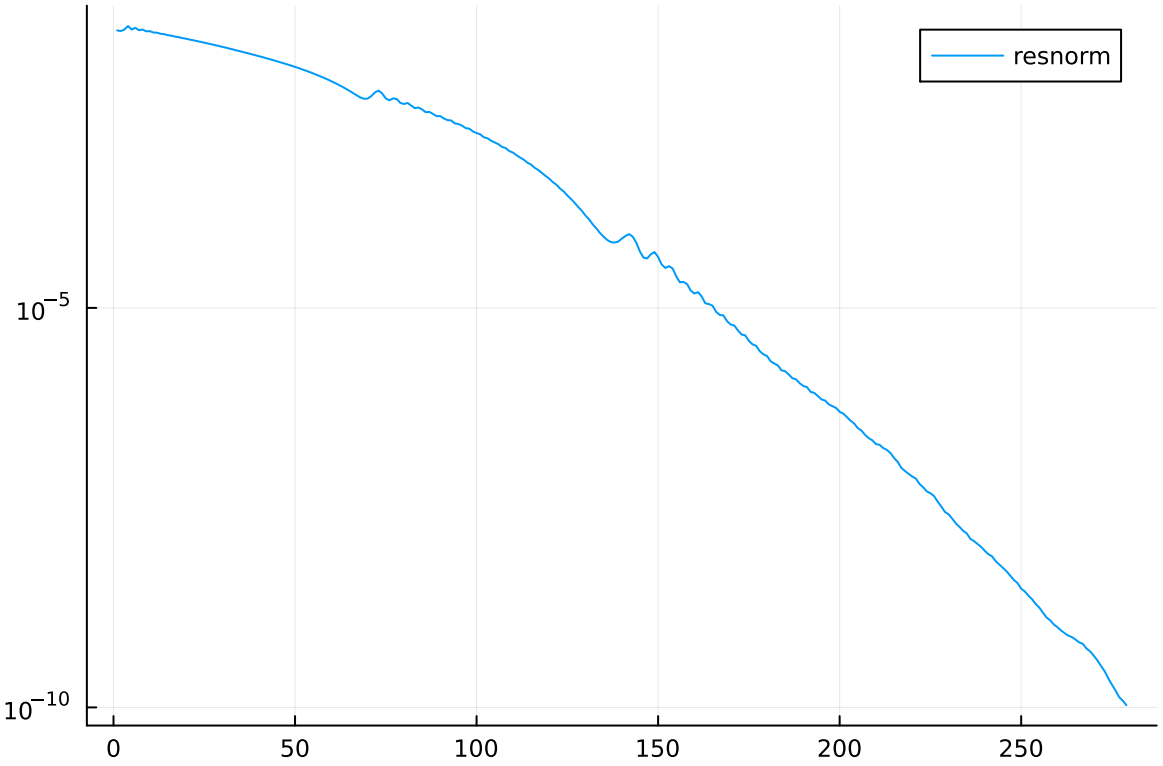
```
demo_problem (generic function with 1 method)
```

```
function demo_problem()
    n = 100
    h = 1/(n+1)
    xx = [i/(n+1) for i=1:n]

    W = zeros(100,100)
    W[50:51,50:51] .= 1000.0
    F = [x^2 + y^2 for x in xx, y in xx]

    h, W, F
end
```

Now we solve this problem using conjugate gradients (CG). Calling the `cg` routine with `log=true` causes the function to return the residual history (`rh`), which we can plot to see the convergence of the method. This isn't exactly terrible convergence, but it also isn't fantastic.

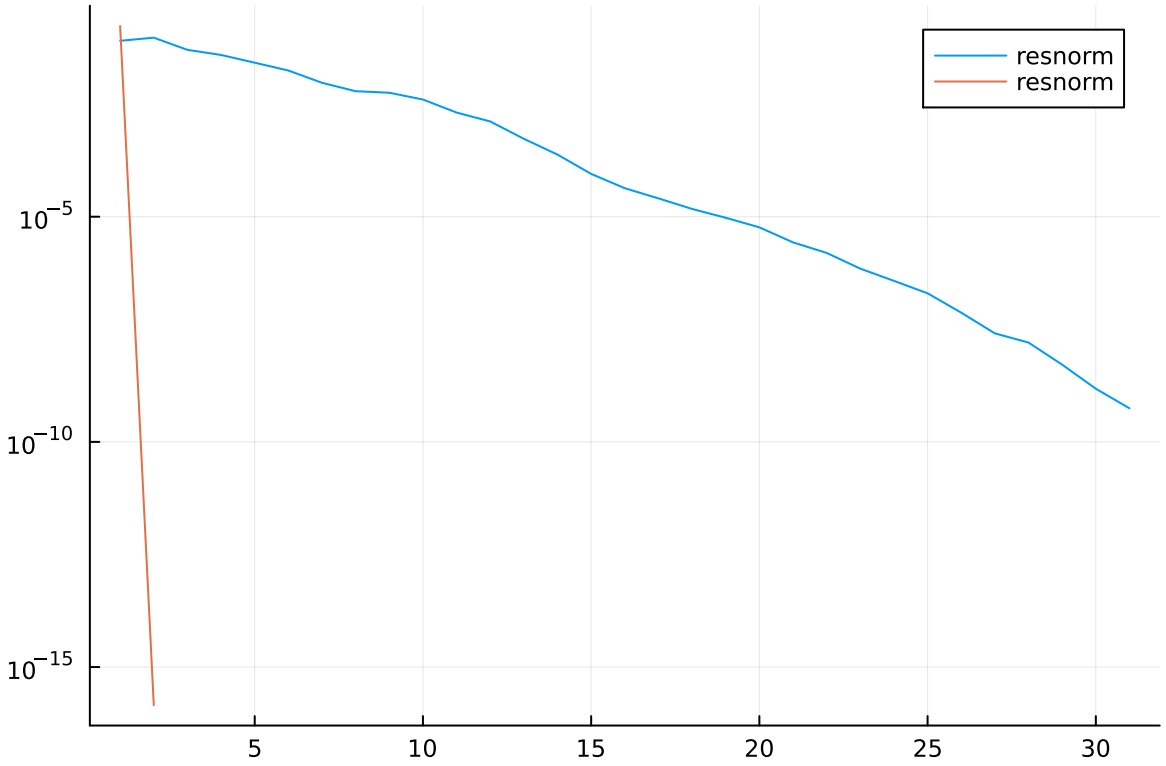



```
let
    h, W, F = demo_problem()
    uu, rh = cg(schrod2db(W), -h^2*F, log=true)
    plot(rh, yscale=:log10)
end
```

Adding a *preconditioner* M causes us to explore Krylov subspaces generated by $M^{-1}A$ and $M^{-1}b$ rather than subspaces generated by A and b. We set up an example preconditioner below to illustrate the interface. The requirement is that we implement `ldiv!(P, x)` for an in-place solve with P, `ldiv!(y, P, x)` for $y = P^{-1}x$, and `P \ x` to compute $P^{-1}x$ and return the result in new storage.

```
begin
    struct MatrixPrecond
        F :: Cholesky
    end
    MatrixPrecond(A :: Matrix) = MatrixPrecond(cholesky(A))
    Base.:\(P :: MatrixPrecond, x) = ldiv!(P.F, copy(x))
    LinearAlgebra.ldiv!(P :: MatrixPrecond, x) = ldiv!(P.F, x)
    LinearAlgebra.ldiv!(y, P :: MatrixPrecond, x) = ldiv!(P.F, copyto!(y, x))
end
```

For a small enough example that this preconditioner makes sense (it uses dense Cholesky, so it won't scale well), we can see that this preconditioner is very effective at reducing the number of steps.




```
let
    n = 10
    h = 1/(n+1)
    xx = [i/(n+1) for i=1:n]

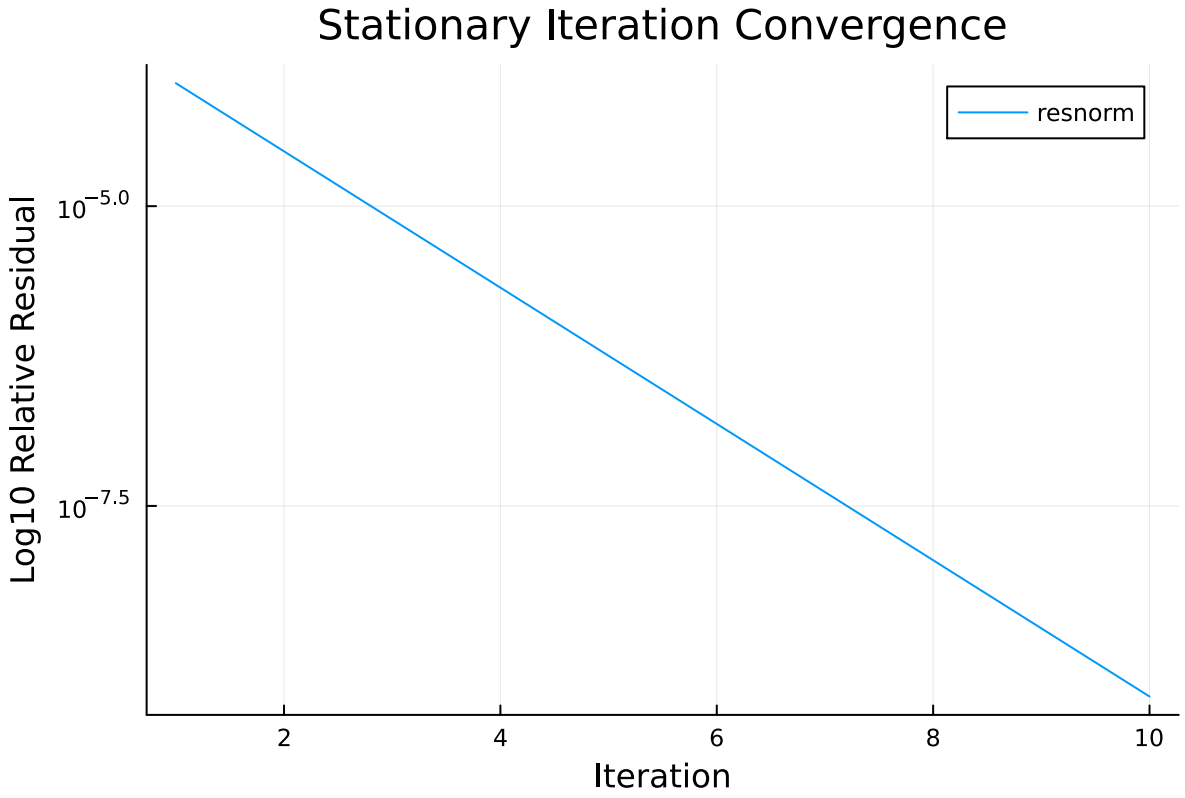
    W = zeros(10,10)
    W[5,5] = 1000.0
    F = [x^2 + y^2 for x in xx, y in xx]

    h = 1/(n+1)
    T = SymTridiagonal(2*ones(n), -ones(n-1))
    In = Matrix(I,n,n)
    P = MatrixPrecond(kron(T,In) + kron(In,T))

    uu0, rh0 = cg(schrod2db(W), -h^2*F, log=true)
    uu1, rh1 = cg(schrod2db(W), -h^2*F, Pl=P, log=true)
    plot(rh0, yscale=:log10)
    plot!(rh1)
end
```

Now it's your turn!

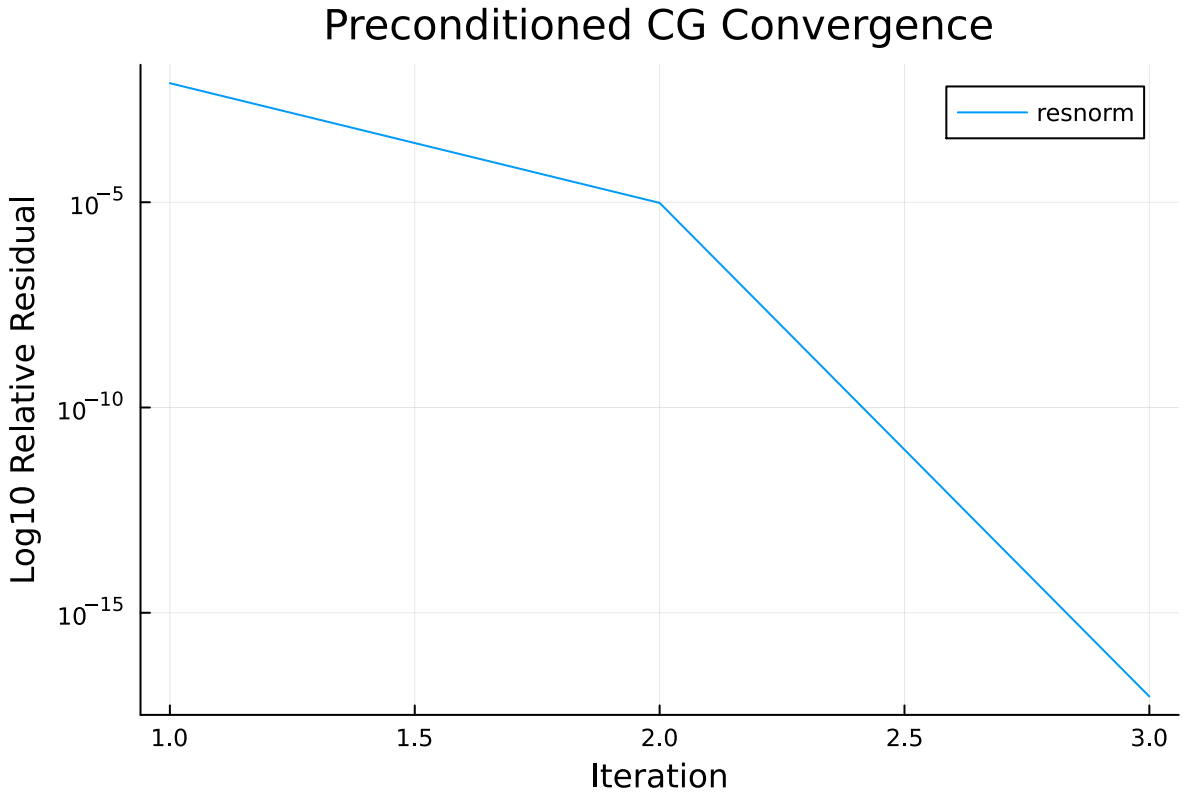
- For the 100-by-100 example from above, plot the convergence for the stationary iteration (as in HW8), thinking of $M^{-1}B$ as solving $TU + UT = B$.
- Plot the convergence of CG preconditioned with M.
- Explain why the preconditioned CG converges in at most $\mathbf{nnz}(W) + 1$ steps, at least in theory. **Hint**: What is the structure of $M^{-1}A$?




```
begin
    h_demo, W_demo, F_demo = demo_problem()

    # Run stationary iteration
    U0 = zeros(100, 100)
    U_stat, resids_stat = schrodinger_iter(W_demo, F_demo, U0, 10)

    # Plot stationary convergence
    plot(resids_stat, yscale=:log10, label="resnorm", xlabel="Iteration",
    ylabel="Log10 Relative Residual", title="Stationary Iteration Convergence")
end
```




```
let
    # Preconditioned CG
    b = -h_demo^2 * vec(F_demo)
    λ = λ1D(100)
    T = SymTridiagonal(2*ones(100), -ones(99))
    In = Matrix(I,100,100)
    P_demo = MatrixPrecond(kron(T,In) + kron(In,T))
    A_demo = schrod2db(W_demo)
    u_cg, ch = cg(A_demo, b; Pl=P_demo, log=true)

    # Plot CG convergence
    plot(ch, yscale=:log10, label="resnorm", xlabel="Iteration", ylabel="Log10
    Relative Residual", title="Preconditioned CG Convergence")
end
```

In case W is dense with all entries positive and nonzero (since $w(x,y) = xy(1-x)(1-y) > 0$ for $x, y \in (0,1)$), so nnz(W) = $n^2 = 10000$. The preconditioned matrix is $M^{-1}A = I + h^2 M^{-1}\,\mathbf{diag}(\mathbf{vec}(W))$, where $\mathbf{diag}(\mathbf{vec}(W))$ is a diagonal matrix with rank equal to the number of nonzero entries, which is 10000 (full rank). Thus, the perturbation $K = h^2 M^{-1}\,\mathbf{diag}(\mathbf{vec}(W))$ has rank 10000. For a matrix of the form $I + K$ with $\mathbf{rank}(K) = r$, the number of distinct eigenvalues is at most $r + 1$, and since the system is SPD, CG converges in at most $r + 1$ steps in exact arithmetic.

However, here $r + 1 = 10001$, which exceeds the system dimension $n^2 = 10000$, so the bound reduces to the trivial ≤ 10000 (full Krylov dimension), making it useless for predicting fast convergence. In practice, convergence is fast not because of low rank, but because the perturbation $h^2 W$ is tiny relative to the spectrum of M (eigenvalues from $\sim 10^{-4}$ to 4), leading to eigenvalues clustered near 1 with small spread ($O(h^2 \max|W|) \approx 10^{-6}$), yielding a low effective condition number.